

TM-V71 Remote

Benutzer- & Technikhandbuch

Kenwood TM-V71 Web-Fernsteuerung

Version 3.2



Web-Fernsteuerung für Kenwood TM-V71(A/E) mit WebRTC-Audio und HackRF-Panadapter, für den Raspberry Pi.

1 Überblick

TM-V71 Remote ist eine moderne, schlanke Web-Fernsteuerung für den Kenwood TM-V71(A/E) Dualband-FM-Transceiver, aufgebaut auf einem direkten seriellen Treiber. Sie bietet volle Gerätesteuerung im Browser, Zwei-Wege-Audio über WebRTC/Opus, vollständige Speicherkanal-Verwaltung, einen optionalen HackRF-Panadapter, klassischen 5-Ton-Selektivruf, einen CW/RTTY/POCSAG-Decoder/Encoder, einen Roh-RX-Rekorder sowie optionale Offline-Rufzeichenerkennung (Vosk). Sie lässt sich als Progressive Web App (PWA) installieren und ist für den Raspberry Pi ausgelegt.

Anders als hamlib (dessen TM-V71-Backends unzuverlässig sind) spricht dieses Projekt den dokumentierten PC-Befehlssatz des Geräts direkt an und erschließt den vollen Funktionsumfang, inklusive der Speicherkanal-Programmierung.

2 Funktionen

- Volle Live-Steuerung beider Bänder (A/B): Frequenz, VFO-/Speichermodus, Relais-Shift & Offset, CTCSS/DCS, Schrittweite, Steuerband, PTT über CAT.
- Speicherkanäle (CHIRP-Niveau): Lesen, Schreiben, Löschen, Umbenennen aller 1000 Kanäle, plus CSV-Import/-Export.
- Live-Status per WebSocket an den Browser; beim Senden leuchtet die UI.
- Zwei-Wege-Audio: direktes WebRTC/Opus zwischen Browser und Backend via aiortc; das Mikrofon speist das Funkgerät nur bei gedrücktem PTT.
- Optionaler HackRF-One-Wasserfall: Echtzeit-Panadapter (folgt der Frequenz) oder Breitband-Sweep.
- Klassischer 5-Ton-Selektivruf (ZVEI-1/2, CCIR, EEA): rufen, dekodieren und RX stummschalten bis zum eigenen Ruf.
- CW (Morse), RTTY (Baudot/AFSK) und POCSAG-Paging (512/1200/2400 Baud, numerisch + alphanumerisch, BCH-FEC) dekodieren + senden über den FM-Audioweg; der CW-Auto-Modus führt Geschwindigkeit und Tonhöhe nach, plus eine Taste zum Offline-Dekodieren eines aufgenommenen RX-Puffers.
- Audioaufbereitung: RX-De-emphasis (für flachen 9600-/Diskriminator-Ausgang), BUSY-gesteuerte Software-Rauschsperrung, TX-AGC und Sprach-Tiefpässe.
- Roh-RX-Rekorder mit WAV-Download (z. B. für ASR-Trainingsdaten).
- Offline-Rufzeichenerkennung (optional, Vosk): erkennt gesprochene deutsche Rufzeichen, prüft sie gegen die BNetzA-Liste (Name/Ort/Klasse bzw. VOID, wenn nicht zugeteilt), Anzeige in der Titelzeile und als Toast.
- Installierbare PWA mit mobilem Querformat-Swipe-Deck.
- Robuster Betrieb: der Handy-Bildschirm bleibt an, das Browser-Audio verbindet sich nach einer Netzstörung automatisch neu, und ein Backend-Watchdog beendet ein eingerastetes PTT, wenn alle Clients verschwinden.
- GPIO-Power-Schalter, Auto-Abschaltung, TX-Leistung, Squelch, S-Meter.
- Zwei Themes (dunkel/hell); kein Build-Schritt für die Oberfläche.

3 Architektur

```
Raspberry Pi
/dev/ttyUSB0 (57600) -- tmv71 driver --+
                                   v
FastAPI backend -- REST + WebSocket (live status)
- control (freq/mode/band/PTT) - memory CRUD + CSV
- CW/RTTY + 5-tone selcall      - HackRF spectrum
- Wavelog logbook (QSO log)     - callsign lookup

USB sound -- aiortc WebRTC <-> browser (Opus, PTT)
FastAPI serves the SPA / PWA at "/" (HTTPS/TLS)
      ^ LAN (HTTPS)
Browser - installable PWA (control + audio)
```

Das Backend besitzt die serielle Schnittstelle direkt (backend/app/tmv71.py). Ein einziger FastAPI-Prozess liefert die SPA/PWA, die REST-Steuerendpunkte, den Live-Status-WebSocket und die WebRTC-Signalisierung — ohne Zusatzdienste. Audio ist intern 48 kHz / 16 Bit / mono (Opus-Standardrate).

4 Voraussetzungen & Hardware

- Raspberry Pi (getestet auf Debian 13 / aarch64), Python 3.11+.
- Kenwood TM-V71(A/E) an einer seriellen Schnittstelle (FTDI-Kabel), 57600 Baud.
- Ein USB-Audiointerface, am Funkgerät verdrahtet (Datenbuchse oder Mic/Speaker), voll duplex.
- Systempakete: portaudio19-dev, swig + liblgpio-dev (optional GPIO).
- Optional: ein HackRF One plus die hackrf-Hosttools für den Wasserfall.
- Optional: vosk + das kleine deutsche Modell (Offline-Rufzeichen-erkennung) sowie pypdf + die BNetzA-Rufzeichenliste-PDF (Name/Ort/Klasse + VOID-Prüfung).

5 Installation

```
sudo apt-get install -y portaudio19-dev python3-venv swig liblgpio-dev
git clone https://github.com/CQ-DJ0SH/tmv71-remote.git
cd tmv71-remote/backend
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt
```

Für einen neustartfesten Betrieb die systemd-Unit aus dem Ordner deploy/ installieren. Der Dienst startet uvicorn mit TLS auf Port 8443.

6 Betrieb über HTTPS

Der Mikrofonzugriff des Browsers (getUserMedia) und der PWA-Service-Worker benötigen einen sicheren Kontext, daher läuft der Server über HTTPS. Ein schnelles selbstsigniertes Zertifikat genügt am Desktop (Warnung einmal bestätigen); für die PWA-Installation auf dem Handy ist ein vertrauenswürdiges Zertifikat nötig (Kapitel 9).

```
cd backend && mkdir -p certs
openssl req -x509 -newkey rsa:2048 -nodes -days 3650 \
  -keyout certs/key.pem -out certs/cert.pem -subj "/CN=tmv71-remote" \
  -addext "subjectAltName=IP:<pi-ip>,DNS:localhost,IP:127.0.0.1"
.venv/bin/uvicorn app.main:app --host 0.0.0.0 --port 8443 \
  --ssl-keyfile certs/key.pem --ssl-certfile certs/cert.pem
```

<https://<pi-ip>:8443/> öffnen und das Zertifikat einmal akzeptieren.

7 Die Weboberfläche

Band-Panels (VFO A / VFO B)

Jedes Band zeigt die Frequenz auf einer 7-Segment-Anzeige mit zwei übereinander liegenden Anzeigen unter einer gemeinsamen S-Skala: einem echten S-Meter (S0-S9) in der Farbe des aktiven Bandes und darunter dem NF-Pegel-/Mikrofon-Modulationsbalken (1 s Peak-Hold). Dazu Bedienelemente für VFO-/Speichermodus, CTRL-/PTT-Bandwahl, TX-Leistung, Squelch (pro Band über Aus-/Einschalten hinweg gespeichert), Relais-Shift/Offset, Ton und Bandbreite. Über die Ziffern-Abstimmung lässt sich jede Stelle per Klick auf die Balken hoch/runter stellen; AIR Band stellt Band A auf das Flugfunkband 118-137 MHz (nur Empfang).

Das S-Meter wird aus dem FM-Quiting abgeleitet: Das Rauschen im oberen Frequenzband des flachen RX-Signals ist umgekehrt proportional zur Signalstärke (lautes Rauschen = kein Signal, volle Rauschunterdrückung = starkes Signal). So lässt sich die Empfangsstärke schätzen, obwohl die TM-V71 über CAT keinen numerischen RSSI liefert — nur einen binären BUSY-Status. Es ist eine relative Quieting-Schätzung: ein verständliches Signal steht im mittleren/oberen Bereich, ein starkes Ortssignal sättigt bei S9, und bei Trägerverlust springt es sofort auf S0 zurück.

PTT & Speicher-Schnellasten

Den großen PTT-Knopf (oder die Leertaste) halten zum Senden; PTT-LOCK rastet den Sendebetrieb ein. PTT und PTT-LOCK setzen verbundenes Audio voraus (sonst gibt es kein Mikrofon) — sie sind deaktiviert, solange Audio aus ist, und werden bei einer Audio-Trennung automatisch beendet. ROGER fügt beim Loslassen einen absteigenden Dreiklang hinzu (d6 · a5 · e5, also 1175/880/659 Hz, je 60 ms mit 30 ms Pause), in der in den Audio-Einstellungen gewählten Lautstärke; während des Sendens zeigt der Knopf einen aufwärts laufenden Timer (MM:SS). Die 1750-Hz-Taste schärft einen Tonruf. Die Speicher-Schnellasten rufen die Kanäle 0-16 ab (M0-M9 in der linken, M10-M16 in der rechten Spalte; die Taste des geladenen Kanals leuchtet); darunter sendet die rechte Spalte drei DTMF-Speicher (0-2). Eine Statuszeile zeigt BUSY je Band, den ASR-Zustand und den Live-RX/TX-Gain (in der PWA; am Desktop steht dort der Sende-Hinweis). Auf dem Handy flankieren Mini-RX/TX-VU-Bars mit Peak-Hold den Knopf.

MUTE und DROP

Zwei Knöpfe flankieren den Schriftzug „PTT → BAND x“ und schalten den Empfangston stumm; sie unterscheiden sich nur darin, wann sie wieder aufhören:

- MUTE (links) ist der schlichte: Er bleibt, bis er erneut gedrückt wird. Die Beschriftung lautet dann MUTED.
- DROP (rechts) blendet die laufende Aussendung aus und hebt sich auf, sobald diese Station die Taste loslässt — für einen Störer oder ein Gespräch, das man nicht mithören möchte, ohne hinterher an das Aufheben denken zu müssen.

Beide leuchten im aktiven Zustand in der Farbe des Sendebands. Drei Punkte sind erwähnenswert:

- DROP gibt auf der fallenden Flanke des BUSY-Signals frei, nicht schon deshalb, weil der

Kanal gerade frei ist: Auf einem stillen Kanal aktiviert, bleibt der Ton bis zum Ende der nächsten Aussendung stumm, statt beim nächsten Statuswechsel sofort wieder aufzugehen.

- DROP gibt außerdem nach drei Sekunden ohne Sprache frei, auch wenn BUSY noch ansteht — der Dauerträger eines Relais hielte den Ton sonst unbegrenzt stumm. Die Schwelle liegt auf dem NF-Pegel: Ein stiller Träger gilt als Stille, eine Aussendung nicht.
- Stummgeschaltet wird im Browser, am Audio-Element. S-Meter, Roh-Rekorder und Rufzeichenerkennung bekommen das Signal weiterhin — still ist nur, was man hört.
- Ein Dreiklang bestätigt jeden Wechsel (e5 · a5 · d6 aufsteigend bei der Freigabe, derselbe Dreiklang rückwärts beim Stummschalten), sodass beides ohne Hinsehen zu unterscheiden ist. Er entsteht im Browser und kann deshalb nie ins Sendesignal geraten — anders als der Roger-Beep, der bewusst dorthin gehört.

Audio (WebRTC/Opus)

Das AUDIO-Panel öffnen, mit dem RX-A/RX-B-Schalter das Empfangsband wählen, CONNECT klicken und das Mikrofon erlauben. RX- und Mic-Pegel werden live angezeigt, dazu die WebRTC-RX/TX-Datenrate in der Graph-Ecke. Bedienelemente: RX/TX-Gain (mit Default-Markierung), MIC (Mic-Test — misst ohne zu tasten, nimmt im Betrieb auf und spielt beim Ausschalten über RX zurück; RX ist dabei stumm), AGC (automatischer TX-Pegel) sowie ein kleiner Rekorder — ● REC / ► PLAY plus WAV-Download des rohen, un-gesquelchten RX-Signals (bis 60 min; z. B. für ASR-Trainingsdaten; der Dateiname des Downloads trägt Datum und Uhrzeit der Sicherung). TX-Timing (Buffer/Trail) und der USB-Mixer liegen unter Einstellungen > Audio. Die Verbindung verbindet sich nach einer Netzstörung automatisch neu und wird beim nächsten Start wiederhergestellt.

RX-Aufbereitung (Einstellungen > Audio): eine RX-De-emphasis (einstellbare Zeitkonstante, standardmäßig an) stellt den natürlichen Klang her, wenn das Audio vom flachen Diskriminator-/9600-Baud-Ausgang kommt; ein fester ~180-Hz-Hochpass im Hörfad entfernt den CTCSS/PL-Subaudioton (67–254 Hz) und das Gleichspannungs-/Brummen, das dieser flache Ausgang durchlässt und die De-emphasis sonst anhebt (hörbar als tiefes Lautsprecherbrummen), ohne die Sprache anzutasten; eine BUSY-gesteuerte Software-Rauschsperrung übernimmt für diesen daueroffenen Ausgang die Stummschaltung aus dem Busy-Status des Geräts; TX/RX-Sprachtiefpässe ($\leq 3,5$ kHz) zähmen Rauschen. Die Decoder erhalten stets das un-gesquelchte, ungefilterte Signal.

Bluetooth-Headsets: Das Sende-Audio wird vom eingebauten Telefon-Mikrofon aufgenommen (nicht vom Headset-Mikro), damit das Headset im A2DP-Profil bleibt und der Empfang in guter Qualität durchkommt. Das Headset-Mikrofon würde Android auf das Mono-Profil HFP/SCO zwingen und RX auf vielen Handys hängen lassen, bis Bluetooth aus/an geschaltet wird.

HackRF-Wasserfall

Ist ein HackRF One angeschlossen, zeigt dieses Panel ein Live-Spektrum über einem Wasserfall: ein Panadapter zentriert auf der Frequenz (folgt automatisch) oder ein Breitband-Sweep. Nur Empfang; LNA/VGA und ein Anzeigepegel sind einstellbar.

Selektivruf (klassisch, 5-Ton)

Klassische Selektivrufe senden und dekodieren (ZVEI-1/2, CCIR, EEA). Einen 5-stelligen

CALL-Code eingeben und CALL drücken (tastet PTT). Den eigenen Code (MY ID) eingeben und MUTE drücken, um RX stumm zu schalten, bis der eigene Ruf empfangen wird — dann wird automatisch entstummt. Über FM ist das AFSK; zum Einstellen einen Dummy-Load verwenden.

Digimodes (CW / RTTY / POCSAG)

Umschalten zwischen CW (Morse), RTTY (Baudot/AFSK) und POCSAG-Paging. DECODE zeigt den empfangenen Text; in das Feld tippen und mit SEND senden (tastet PTT); die CW-Eingabe wird in Großbuchstaben erzwungen. Parameter: CW WpM/Tonhöhe — mit AUTO-Modus, der Geschwindigkeit und Tonhöhe nachführt (live auf den Slidern) und Sprache/Rauschen verwirft, sodass er auch nach einer Sprachansage auf das CW einrastet; RTTY Baud/Shift/Mark; sowie POCSAG-Baud (512/1200/2400), RIC, Funktion und numerisch/alphanumerisch (RX auto-erkannt) — z. B. DAPNET auf 439,9875 MHz mit RIC/FUNC/Zeitstempel pro Meldung. Die REC-Taste dekodiert den Roh-RX-Puffer offline im aktuellen Modus. Über das FM-Gerät ist das MCW / AFSK / FSK — keine echten HF-Modes.

Rufzeichenerkennung (Vosk)

Eine optionale, Offline-Spracherkennung auf dem RX-Audio, die gesprochene deutsche Rufzeichen erkennt; einzuschalten unter Einstellungen > Audio. Ein grammatik-beschränktes Vosk-Modell (ITU/NATO-Buchstabieralphabet plus deutsche Ziffern — die deutsche Buchstabiertafel und die Buchstabennamen wurden entfernt, da ihre kurzen, homophon-anfälligen Wörter die meisten Falschtreffer verursachten) bleibt auf verrauschter FM-Sprache brauchbar; die erkannten Zeichen werden zu einem Rufzeichen gefügt, auf die realen deutschen BNetzA-Präfixblöcke beschränkt (immer 5–6 Zeichen) und gegen die BNetzA-Rufzeichenliste geprüft. Die Genauigkeit steigt durch N-Best-Rescoring — Vosk liefert mehrere Hypothesen pro Durchgang, und das beste Rufzeichen daraus wird gewählt, vorzugsweise ein zugeteiltes — sowie durch Wiederholungs-Voting: ein gelistetes Rufzeichen wird sofort angezeigt, ein nicht zugeteiltes (VOID) erst, wenn es zweimal in kurzer Zeit gehört wurde, was einmalige Fehlhörer unterdrückt (OMs geben ihr Call ohnehin 2–3×). Ein Treffer erscheint in einem umrahmten Feld in der Titelzeile (in Bandfarbe) und als Toast, angereichert aus der Offline-Liste mit Name, Ort und Klasse (A/E/N); ein nicht zugeteiltes Rufzeichen wird dennoch angezeigt, aber als VOID markiert. Das eigene Rufzeichen wird ignoriert, es läuft nur bei offener Rauschsperrung und kann auch das Mic-Test-Audio auswerten. Die Rufzeichenliste wird einmalig per Converter aus der PDF erzeugt (python -m app.callsign_list); QRZ.com wird nur bei der manuellen Abfrage im Logbuch genutzt, nie von der ASR.

Wird das Funkgerät abgeschaltet, setzt die Erkennung aus — es gibt kein RX-Audio zu analysieren — und nimmt beim Einschalten wieder auf. Die gespeicherte Einstellung wird dabei nicht überschrieben, die Erkennung bleibt nach einem Aus- und Einschalten also nicht unbemerkt abgeschaltet. ASR-Anzeige und Rufzeichenfeld bleiben durchgehend sichtbar; nur die LED wechselt: grün pulsierend im Betrieb, rot während des Aussetzens, gedimmt bei ausgeschalteter Erkennung. Der Puls bedeutet „hört gerade zu“ — deshalb pulst keiner der beiden Aus-Zustände.

Bandscan

Einen VHF/UHF-Bereich oder die Speicherbank absuchen und ein Belegungs-Spektrum +

Wasserfall sehen. Ein Doppelklick auf einen Kanal stimmt den Steuer-VFO darauf ab.

ASR-Kontakte

Ein Panel unter dem Bandscan, das jede erkannte Station als Karteikarte sammelt — die letzten Durchgänge sind so auf einen Blick lesbar statt als durchlaufendes Protokoll. Die Karten liegen in einem Kartenkasten aus leeren Fächern, die neueste vorn. Die letzten 200 Einträge hält der Pi und stellt sie beim Öffnen des Panels wieder her; CLEAR ALL leert die Ansicht. Im mobilen Deck hat das Panel keinen Tab — dorthin hinter dem Bandscan Wischen.

Jede Karte trägt:

- Das Rufzeichen, in der größten und fettesten Schrift der Karte — es ist die Identität des Kontakts. Die Null wird durchgestrichen dargestellt (DJØSH), damit sie nicht als Buchstabe O gelesen wird; das gilt nur für die Anzeige, ins Logbuch geht weiterhin die schlichte 0.
- Einen Avatar, dessen Buchstaben und Farbe aus dem Rufzeichen selbst abgeleitet sind (die Zeichen nach der Regionalziffer, dazu ein Farbton aus einem Hash des ganzen Rufzeichens). Eine Station sieht damit in jeder Sitzung gleich aus, ohne dass etwas gespeichert wird.
- Alle drei deutschen Lizenzklassen A / E / N, wobei nur die des Inhabers leuchtet und die beiden anderen gedimmt bleiben. Leuchtet keine, steht das Rufzeichen nicht in der BNetzA-Liste.
- Name und Ort des Inhabers aus der Offline-Liste der BNetzA — nie von QRZ.com, das die ASR nicht abfragt.
- Die Uhrzeit der letzten Nennung und die Redezeit der Station, fest an der Unterkante der Karte.

Eine erneut gehörte Station bekommt keine zweite Karte. Die vorhandene leuchtet auf, wird markiert und zählt hoch (×2, ×3 ...) — sie bleibt aber an ihrem Platz, damit sich der Kartenkasten nicht bei jedem Durchgang unter dem Blick umsortiert. Genau eine Karte trägt die rote Umrandung: die zuletzt gehörte. Die Reihenfolge richtet sich damit nach dem Erstkontakt, nicht nach der letzten Nennung.

Beim Überfahren einer Karte erscheinen die Erkennerdetails, die früher die Protokollzeilen füllten: die Wort-Konfidenz (0.00–1.00, Mittelwert über die einzeln buchstabierten Zeichen — eine Aussage über die Akustik, nicht darüber, ob es das Rufzeichen wirklich gibt), S-Wert und Empfangsband zum Zeitpunkt der Erkennung, der von Vosk tatsächlich gehörte Rohtext, die verworfenen N-Best-Kandidaten sowie eine gegebenenfalls angewandte 5/6-Zeichen-Korrektur.

Zwei Knöpfe je Karte: Das Wiedergabesymbol trägt das QSO direkt ins Wavelog ein, mit dem Namen aus der BNetzA-Liste vorbelegt, und färbt sich nach erfolgreicher Übertragung türkis, damit nichts zweimal gesendet wird. Das Kreuz entfernt einen falsch erkannten Kontakt. Diese Löschung geschieht auf dem Pi, nicht nur im Browser: Die Karte fällt aus dem Protokollpuffer (sonst käme sie beim nächsten Öffnen des Panels zurück), alle verbundenen Clients verlieren sie gleichzeitig, und das Rufzeichen wird aus dem 90-Sekunden-Dedupe-Fenster entlassen, damit eine korrigierte Erkennung sofort wieder gemeldet werden darf.

Redezeit. Die Uhr in der Fußzeile einer Karte ist der Start/Stopp-Knopf für den Zeitzähler

dieser Station — und der Zähler läuft auch von allein: Er startet auf der steigenden Flanke von BUSY und hält auf der fallenden, ein Durchgang wird also ohne Zutun erfasst. Die Uhr gehört immer genau einer Karte, der markierten. Ein Klick auf eine Karte verschiebt die Markierung und mit ihr den Zähler dorthin — für den Fall, dass die Erkennung einen Durchgang der falschen Station zugeschrieben hat. Jede Karte führt ihre eigene Summe: Beim Fokuswechsel wird der laufende Abschnitt auf der bisherigen Karte verbucht und der Wert der neuen übernommen, es geht also nichts verloren und nichts wird doppelt gezählt. Die Anzeige steht fest auf hh:mm:ss, sonst änderte der Knopf im Zählen seine Breite.

Die Titelzeile des Panels trägt die Summe aller Zähler (Σ) und fünf Bedienelemente:

- Das Eingabefeld nimmt ein Rufzeichen von Hand auf, wenn die Erkennung eines verpasst — nur Großbuchstaben und Ziffern, bereits beim Tippen gefiltert. LOG (oder die Eingabetaste) legt die Karte über das Backend an, genau wie bei einer erkannten Station: Sie landet in der Historie und erreicht alle verbundenen Clients. Eine Meldung sagt, ob die BNetzA-Liste das Rufzeichen kannte.
- MOD kennzeichnet die markierte Karte als Moderator. Deren Zeit läuft weiter und bleibt auf der Karte ablesbar, wird aber nirgends gewertet: weder in der Σ -Summe noch in der Rangliste, und sie bestimmt auch nicht den Maßstab der Balken — sonst drückte die Station, die die Runde zusammenhält, alle anderen Balken flach. Eine gekennzeichnete Karte trägt einen Ring um den Avatar. Die Kennzeichnung liegt im Browser (Local Storage), nicht auf dem Pi: Sie gehört dem, der diese Runde mithört, und übersteht einen Reload.
- STATS öffnet die Auswertung: alle Karten nach Redezeit, die längste zuerst, mit Rufzeichen, Name, Balken und Zeit. Der Balken bezieht sich auf die längste Redezeit, nicht auf die Summe. Moderatoren stehen ohne Balken und ohne Rang unter der Rangliste. COPY legt die Liste als Klartext in die Zwischenablage. Der Dialog zählt weiter, solange eine Station zu hören ist, kann also über einen Durchgang hinweg offen bleiben.
- CLEAR ALL leert den Kartenkasten — in Dunkelrot, weil es alle Karten auf einmal betrifft. Es räumt nur die Ansicht; das $\times$ auf einer einzelnen Karte löscht den Kontakt zusätzlich auf dem Pi.
- VOICE am rechten Ende schaltet die Stimmerkennung ein und aus (siehe unten). Die Lampe pulst grün, solange sie zuhört; ein umrandeter Knopf bedeutet, dass sie nur beobachtet, ein gefüllter, dass sie eingreift.

Rufzeichen korrigieren: auf das Rufzeichen der Karte klicken und überschreiben (Eingabetaste übernimmt, Escape und Wegklicken verwerfen). Die Karte behält Redezeit, Zähler und Platz, Name und Ort werden für das korrigierte Rufzeichen neu gelesen, und die Änderung erreicht jeden offenen Client. Hat das korrigierte Rufzeichen schon eine Karte, werden beide verschmolzen und ihre Zeiten addiert. Das Stimmprofil zieht mit um — sonst sammelte ein verhörtes Rufzeichen weiter einen Stimmabdruck unter einem Namen, den es nie gab, während die wirkliche Station nie einen bekommt.

Stimmerkennung — ein Durchgang ohne Rufzeichen

Nicht in jedem Durchgang fällt ein Rufzeichen, und die Redezeit zählt dann auf die zuletzt markierte Karte. Die Sprechererkennung schließt diese Lücke: Der Stimmabdruck eines Durchgangs wird mit den Stationen verglichen, deren Rufzeichen schon einmal zu hören war. Das Einlernen kostet nichts — jeder erkannte Ruf beschriftet seine eigene Aufnahme,

die Profile bauen sich also im laufenden Betrieb auf.

- Ein Durchgang endet an der SPRECHPAUSE, nicht am Träger. Über ein Relais bleibt BUSY die ganze Verbindung stehen, die fallende Flanke kommt also nie, und alle Stationen lägen in einem Segment mit zu Brei gemittelten Stimmen. Die Pausengrenze trägt auch im Simplexbetrieb, wo der Trägerabfall schlicht früher kommt.
- Unter etwa 3 s Sprache wird nichts geraten, der Durchgang entfällt. Ebenso ein Durchgang, in dem zwei Rufzeichen fielen — ihn zu beschriften würde beide Profile verderben.
- Zwei Stufen. Beobachten entscheidet und meldet nur — das Urteil steht über dem Kartenkasten und in den Detailangaben der Karte, bewegt wird nichts. Zuordnen verschiebt zusätzlich Marke und Redezeit auf die erkannte Station; eine solche Karte wird gestrichelt umrandet und ist damit nie mit dem durchgezogenen Rot von „hat gerade sein Rufzeichen genannt“ zu verwechseln. Die Stufe wird unter Einstellungen > Audio > Voice ID gewählt.
- Eine VON HAND gewählte Kachel hat immer Vorrang: Solange sie gilt, verschiebt die Stimme weder Marke noch Timer. Sie wird mit dem Beginn des nächsten Durchgangs frei, die Korrektur gilt also für den Durchgang, in dem sie gemacht wurde. Ein tatsächlich gehörtes Rufzeichen verschiebt die Marke weiterhin — das ist Beleg, keine Vermutung.
- Sie setzt die eingeschaltete Rufzeichenerkennung voraus (sie hört am selben Audio-Abgriff mit) und kostet je Durchgang etwa 1,3 s auf einem Kern, im Anschluss.

Gemessen an zwei Mitschnitten derselben Runde (14 + 11 Durchgänge, vier Stationen in beiden): dieselbe Station über zwei Aufnahmen hinweg 0,62–0,79 Kosinus, verschiedene Stationen im Mittel 0,33 und im schlechtesten Fall 0,64. Bei Schwelle 0,55 mit 0,05 Abstand zum Zweitplatzierten wurden 3 der 4 bekannten Stationen erkannt, keine falsch, und keine der 7 dem Profilbestand fremden Stationen mit einer bekannten verwechselt. Der Abstand leistet dabei die eigentliche Arbeit: Bei einer fremden Stimme liegt der beste Treffer flach vorn (0,01–0,08), bei einer echten deutlich (0,20–0,34). Also ein Assistent, der sich enthalten darf — kein Beweis, und nie Grundlage für einen Logbucheintrag allein.

Das Modell ist Vosks eigenes Sprechermodell (vosk-model-spk-0.4, ~14 MB), ein X-Vektor, trainiert auf 8-kHz-Telefonsprache: schmalbandig, verrauscht, kanalvariabel — und damit dem FM-Signal über ein Relais viel näher als ein Breitbandmodell. Ein Stimmabdruck ist ein biometrisches Merkmal: Die Profile bleiben in speakers.json auf dem Pi (gitignored, kein Upload), und mit der Karte verschwindet auch das Profil.

Logbuch (Wavelog + QRZ.com)

Protokolliert QSOs in eine lokal installierte Wavelog-Instanz. Es genügt, das Rufzeichen (und optional einen Namen) einzugeben — Frequenz, Band, Modus, Datum/Uhrzeit und das eigene Rufzeichen werden automatisch aus dem aktuellen Steuerband und dem Stationsprofil ergänzt. LOOKUP holt Name, Locator, QTH, Land und E-Mail von QRZ.com (XML-Daten-API) sowie 'schon gearbeitet' / DXCC von Wavelog. LOG QSO überträgt den Kontakt als ADIF. Ein grüner Punkt zeigt, ob Wavelog erreichbar ist; das Panel listet die letzten QSOs (mit den ermittelten Details, einzeln oder per CLEAR löscher) sowie die QSO-Zähler von Wavelog (heute/Monat/Jahr/gesamt). Wavelog-URL, API-Token und Stationsprofil sowie QRZ.com-Benutzer/Passwort werden unter Einstellungen > Logging hinterlegt. Zugangsdaten liegen nur auf dem Pi (runtime.json) und werden nie committet.

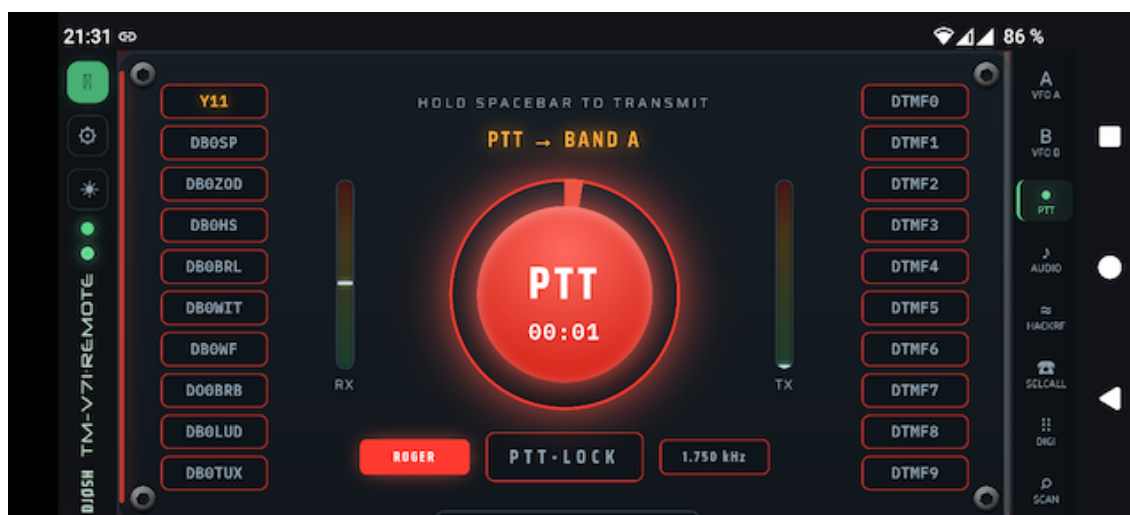
Einstellungen

Reiter: Allgemein (Rufzeichen, API-Backend-URL, serieller Port/Baud, GPIO-Power, Auto-Abschaltung, Logo, GitHub-Update, Root-CA-Download), Audio (Gerät, USB-Mixer, Sprachfilter, Testton, TX-Timing), Rig-Info, Rig-Speicher, Rig-DTMF, Logging (Wavelog + QRZ.com) und Pi-Hardware (Host-Metriken).

8 Mobile App (PWA)

Die Oberfläche installiert sich als Progressive Web App: Vollbild, mit App-Shell-Service-Worker für sofortigen Start. Auf dem Handy werden die Panels zu einem vertikalen Swipe-Deck — nach oben/unten Wischen, ein Panel pro Bildschirm (so liegt die Scroll-Achse des Decks nicht auf den waagerechten Schieberegler, die dadurch bedienbar bleiben) —, die Titelzeile zu einer schmalen vertikalen Leiste links und die Tab-Leiste rechts. Hinter dem letzten Panel folgt eine Info-Seite mit App-Version und Browser-/Umgebungsdaten. Die App wird ins Querformat gezwungen; im Hochformat erscheint ein Dreh-Hinweis. Installation über das Browser-Menü (Installieren / Zum Startbildschirm); unter iOS über Safari > Teilen.

Solange die App geöffnet ist, bleibt der Handy-Bildschirm über die Screen-Wake-Lock-API wach und schaltet sich nicht mitten im QSO ab. Die Browser-Audioverbindung verbindet sich nach einer kurzen Netzstörung automatisch neu, und geht die Verbindung zum Backend bei eingerastetem Sendebetrieb verloren, wird das PTT beendet (lokal und durch einen Backend-Watchdog) — das Gerät kann so nie unbeaufsichtigt getastet bleiben.



9 Vertrauenswürdiges Zertifikat (Root-CA)

Ein selbstsigniertes Zertifikat genügt am Desktop, aber mobile Browser installieren die PWA nicht und starten den Service-Worker nicht ohne vertrauenswürdiges Zertifikat. Eine eigene Root-CA erstellen, das Serverzertifikat damit signieren und die CA auf dem Handy als vertrauenswürdige installieren. Ein Download-Link erscheint unter Einstellungen > Allgemein, sobald die CA existiert.

```
cd backend/certs && mkdir -p ca
# 1) Root CA (10 years) – keep ca.key secret
openssl genrsa -out ca/ca.key 4096
openssl req -x509 -new -key ca/ca.key -sha256 -days 3650 \
  -subj "/CN=TM-V71 Remote Root CA" \
  -addext "basicConstraints=critical,CA:TRUE,pathlen:0" \
  -addext "keyUsage=critical,keyCertSign,cRLSign" -out ca/ca.crt
# 2) server cert signed by the CA (list every name/IP in the SAN)
openssl genrsa -out key.pem 2048
openssl req -new -key key.pem -subj "/CN=tm-v71.example.lan" -out s.csr
openssl x509 -req -in s.csr -CA ca/ca.crt -CAkey ca/ca.key \
  -CAcreateserial -days 825 -sha256 -extfile leaf.ext -out cert.pem
sudo systemctl restart tmv71-remote.service
```

ca/ca.crt auf dem Handy installieren (Einstellungen > Sicherheit > Zertifikat installieren > CA-Zertifikat). Das Leaf ist 825 Tage gültig und kann ohne Neu-Import aus derselben CA erneuert werden. ca/ca.key geheim halten.

10 Konfiguration

Einstellungen kommen aus Umgebungsvariablen (Präfix TMV71_) oder einer .env-Datei; Änderungen aus der Web-UI werden in backend/app/runtime.json gespeichert. Wichtige Variablen:

```
TMV71_SERIAL_PORT=/dev/ttyUSB0  TMV71_SERIAL_BAUD=57600
TMV71_HOST=0.0.0.0              TMV71_PORT=8443
TMV71_AUDIO_DEVICE=NAD          TMV71_AUDIO_ENABLED=true
TMV71_GPIO_POWER_PIN=17         TMV71_CALLSIGN=DJ0SH
TMV71_ASR_MODEL_DIR=...         TMV71_ASR_CALLLIST_PDF=...
TMV71_SSL_CERTFILE=...          TMV71_SSL_KEYFILE=...
```

11 REST- & WebSocket-API

Steuerung & Status

GET	/api/status	live radio state
POST	/api/frequency	set VFO frequency
POST	/api/band-mode	VFO / memory / call
POST	/api/control-band	select control band
POST	/api/ptt	key / un-key (CAT)
POST	/api/ptt-band	select TX band
POST	/api/squelch /step	squelch level / step
POST	/api/vfo	shift/offset/tone/bw
GET	/api/info /version	rig + app info
WS	/ws	live status stream

Speicherkanäle

GET	/api/memories?start&end	list channels
GET	/api/memories/{ch}	one channel
PUT	/api/memories/{ch}	write channel
DELETE	/api/memories/{ch}	clear channel
GET	/api/memories.csv	export CSV
POST	/api/memories/import	import CSV
POST	/api/recall	recall to band

Audio

POST	/api/webrtc/offer	WebRTC SDP offer
GET	/api/audio/status	RX/TX levels, flags
GET	/api/audio/devices	list sound devices
POST	/api/audio/device	pick device
POST	/api/audio/gain	rx/tx gain + TX AGC
POST	/api/audio/buffer	tx buffer / ptt tail
POST	/api/audio/tones	roger/test/mic/lowpass/de-emph/squelch
POST	/api/audio/record[/clear]	raw RX recorder
GET	/api/audio/record.wav	download recording (WAV)
GET/POST	/api/audio/mixer	USB card mixer

Digimodes & Selektivruf

GET	/api/digi	CW/RTTY/POCSAG status
POST	/api/digi/config	mode + parameters
POST	/api/digi/tx	encode + transmit
POST	/api/digi/decode-recording	decode the RX buffer
WS	/ws/digi	decoded text stream
GET/POST	/api/asr/config	callsign recognition (Vosk)
POST	/api/asr/log	add a callsign by hand
PATCH	/api/asr/log/{call}	correct a callsign (profile follows)
GET/POST	/api/asr/speaker	voice ID: on/off, stage, limits
DELETE	/api/asr/log/{call}	drop a misrecognised contact
WS	/ws/callsign	recognised-callsign events
GET	/api/selcall	5-tone status
POST	/api/selcall/config	standard / tone / own
POST	/api/selcall/tx	send a 5-tone call

WS	/ws/selcall	decoded calls stream
----	-------------	----------------------

SDR & Scan

GET	/api/hackrf	SDR status
POST	/api/hackrf/start stop config	
WS	/ws/hackrf	spectrum/waterfall frames
GET	/api/scan	POST /api/scan/start stop band scan

Logbuch

GET/POST	/api/log/config	logbook credentials (Wavelog + QRZ)
POST	/api/log/test	test the Wavelog connection
POST	/api/log/qrz/test	test the QRZ.com login
GET	/api/log/stations	Wavelog station profiles
POST	/api/log/lookup	callsign lookup (QRZ + Wavelog)
POST	/api/log/qso	log a QSO
GET	/api/log/recent	recent QSOs + online + Wavelog stats
POST	/api/log/recent/delete	delete one recent entry
POST	/api/log/recent/clear	clear the recent list

System & Power

GET/POST	/api/power-switch	GPIO power
POST	/api/gpio-config	set GPIO pin
POST	/api/auto-power-off	idle auto-off
GET/POST	/api/serial-config	serial port/baud
GET/POST	/api/callsign /theme	
GET	/api/system	Pi host metrics
GET/POST	/api/update	GitHub self-update

12 Fehlerbehebung

- Kein CAT / 'radio offline': Port und Baud in den Einstellungen prüfen; das FTDI-Kabel muss auf /dev/ttyUSB0 liegen (oder Port korrekt setzen).
- Kein Audio: HTTPS sicherstellen, CONNECT klicken und Mic erlauben; USB-Karte und Mixer prüfen (Playback treibt das Funk-Mic beim Senden).
- PWA installiert nicht / Theme schaltet am Handy nicht: Zertifikat ist nicht vertrauenswürdig — Root-CA installieren (Kapitel 9).
- RX-Filter nur auf einem Kanal: Audio neu verbinden (Disconnect/Connect), damit Opus auf Mono neu ausgehandelt wird.
- Decoder brauchen ein sauberes Signal; Pegel und (bei RTTY) den Mark-Ton anpassen.

13 Sicherheit

Nur fürs LAN konzipiert; es gibt keine Authentifizierung. Den Port nicht direkt ins Internet stellen — ein VPN (WireGuard/Tailscale) oder einen Reverse-Proxy mit TLS + Auth verwenden. Der private CA-Schlüssel verlässt den Pi nie.

14 Danksagung & Lizenz

Kenwood-PC-Protokoll-Doku: LA3QMA/TM-V71_TM-D710-Kenwood. Erstellt mit aiortc (WebRTC/Opus) und sounddevice. Schriften: Saira, IBM Plex Mono, DSEG (7-Segment), Neuropol (Titel). Lizenzdetails im Repository.